

Projets Pratiques à Réaliser

18 projets concrets du capteur au cloud — niveaux débutant, intermédiaire, avancé
Couvrant les 4 blocs de compétences SNI : Acquisition · Signal · Commande · Embarqué

Pourquoi faire des projets matériels personnels ?

Les recruteurs en systèmes embarqués et IoT cherchent des candidats capables de **faire fonctionner quelque chose de réel**. Un proto qui clignote une LED en réponse à un capteur de température dit plus sur toi qu'un cours magistral. Ces projets couvrent toute la chaîne SNI : du capteur physique au traitement logiciel, du schéma électronique au code embarqué, de la commande au cloud.

Matériel de base recommandé (budget ~80–150€) : Raspberry Pi 4 (35€), Arduino Uno ou ESP32 (10€), kit capteurs variés (15€), oscilloscope USB (30€), breadboard + composants (10€).

■ NIVEAU 1 — Débutant (S1–S2)

■ NIVEAU 2 — Intermédiaire
(S3–S4)

■ NIVEAU 3 — Avancé (S5–S6)

■ BLOC 1 — Concevoir un Système d'Acquisition

Projet #01	Station météo avec Arduino et capteurs multiples	■ 3 semaines
Contexte	Conçois une mini-station météo mesurant température, humidité, pression et luminosité avec un Arduino Uno. Affiche les valeurs sur un écran LCD et enregistre les données sur carte SD.	
Objectifs	• Câbler capteurs DHT22, BMP280, LDR selon datasheets • Programmer en C++ Arduino (I2C, SPI, UART) • Calibrer et vérifier les mesures avec références • Structurer les données (CSV sur SD card)	
Stack	Arduino Uno, DHT22, BMP280, écran LCD I2C, module SD	
Livrable	Prototype fonctionnel + code commenté + rapport de calibrage	
■ Bonus	Ajouter transmission Wi-Fi vers Raspberry Pi via ESP8266	
ACs SNI	AC 1.1, 1.4, 1.7, 1.8, 1.9	

Projet #02	Voltmètre/oscilloscope USB avec Python	■ 2–3 semaines
Contexte	Utilise une carte Arduino comme ADC pour échantillonner un signal analogique (audio ou électrique) et visualise le spectre en temps réel depuis Python via port série.	
Objectifs	• Comprendre l'échantillonnage (théorème de Shannon) • Programmer l'ADC Arduino à fréquence contrôlée • Transférer via UART série vers Python • Afficher signal temporel et FFT avec Matplotlib	
Stack	Arduino Uno, Python (PySerial, NumPy, Matplotlib), signal générateur audio	
Livrable	Application Python avec oscilloscope + analyseur de spectre temps réel	
■ Bonus	<i>Comparer avec un vrai oscilloscope et mesurer les erreurs</i>	
ACs SNI	AC 1.6, 1.8, 1.9, AC 2.2, 2.3	

Projet #03	Réseau de capteurs sans fil avec LoRaWAN	■ 5 semaines
Contexte	Déploie 3 nœuds ESP32 équipés de capteurs (température, CO2, humidité). Transmets les données via LoRaWAN vers un gateway TTN (The Things Network) puis stocke dans InfluxDB et visualise dans Grafana.	
Objectifs	• Configurer modules LoRa (SX1276) et protocole TTN • Optimiser la consommation (mode deep sleep entre transmissions) • Décoder les payloads avec Node-RED ou Python • Créer tableau de bord Grafana	
Stack	ESP32 + SX1276, TTN (The Things Network), Node-RED, InfluxDB, Grafana	
Livrable	Réseau de 3 nœuds + dashboard temps réel + rapport de consommation	
■ Bonus	<i>Calcul d'autonomie sur batterie et optimisation duty cycle</i>	
ACs SNI	AC 1.12, 4.12, 4.13	

■ BLOC 2 — Traiter des Données Issues de Capteurs

Projet #04	Analyse fréquentielle d'un signal audio (Python)	■ 2–3 semaines
Contexte	Enregistre différents signaux audio (voix, instruments, bruit) avec un micro. Implémente des filtres numériques FIR et IIR, analyse les spectrogrammes, compare les méthodes de fenêtrage.	
Objectifs	• Implémenter FFT et STFT (fenêtrage Hanning, Hamming) • Concevoir filtres FIR passe-bas et IIR Butterworth • Visualiser spectrogrammes avec SciPy + Matplotlib • Mesurer la réponse impulsionnelle	
Stack	Python (NumPy, SciPy, Matplotlib, Soundfile), micro USB	
Livvable	Notebook Python avec analyse complète + rapport de comparaison filtres	
■ Bonus	<i>Déployer un filtre FIR en temps réel sur STM32 avec CMSIS-DSP</i>	
ACs SNI	AC 2.1, 2.2, 2.3, 2.4	

Projet #05	Détection d'anomalies vibratoires (maintenance prédictive)	■ 6 semaines
Contexte	Place un accéléromètre (ADXL345 ou MPU6050) sur un moteur ou ventilateur. Acquiert les vibrations, applique FFT, entraîne un modèle ML pour détecter les anomalies annonçant une panne.	
Objectifs	• Acquérir signaux vibratoires à haute fréquence (kHz) • Extraire features spectrales (RMS, peak frequency, kurtosis) • Classifier normal/anomalie avec SVM ou Random Forest • Déployer détection en temps réel sur Raspberry Pi	
Stack	Raspberry Pi, MPU6050 (I2C), Python (SciPy, scikit-learn), Grafana	
Livvable	Système de détection d'anomalie + rapport d'évaluation du modèle	
■ Bonus	<i>Entraînement TinyML sur microcontrôleur (Edge Impulse)</i>	
ACs SNI	AC 2.7, 2.8, 2.12, 2.15, 2.16	

Projet #06	Traitement et analyse d'images satellites (télédétection)	■ 5 semaines
Contexte	Télécharge des images satellites Sentinel-2 (service Copernicus, gratuit). Calcule des indices de végétation (NDVI), segment les zones urbaines, forêts et champs. Analyse l'évolution au cours du temps.	
Objectifs	• Télécharger et prétraiter des images multi-spectrales • Calculer NDVI, NDWI avec NumPy • Segmentation supervisée avec Random Forest • Visualiser l'évolution temporelle	
Stack	Python (GDAL, Rasterio, scikit-learn, Matplotlib), données Sentinel-2 (Copernicus)	
Livrable	Pipeline d'analyse + cartes thématiques + rapport scientifique	
■ Bonus	<i>Mesurer les conséquences du changement climatique sur une zone cible</i>	
ACs SNI	AC 2.9, 2.10, 2.16, 2.17	

Projet #07	Vision par ordinateur embarquée (détection d'objets temps réel)	■ 8 semaines
Contexte	Déploie un modèle de détection d'objets YOLOv8 nano sur Raspberry Pi 4 ou Jetson Nano. Détecte des objets en temps réel depuis une caméra USB. Optimise pour atteindre 10+ FPS.	
Objectifs	• Entraîner ou fine-tuner YOLOv8 nano sur dataset personnalisé • Optimiser le modèle (quantification INT8, TFLite) • Déployer sur Raspberry Pi avec OpenCV • Mesurer et optimiser les performances (FPS, latence)	
Stack	Raspberry Pi 4 ou Jetson Nano, Caméra USB, Python, YOLOv8, TFLite	
Livrable	Système de détection embarqué + benchmark performances	
■ Bonus	<i>Intégrer une alerte MQTT vers un serveur central si objet détecté</i>	
ACs SNI	AC 2.10, 2.11, 2.16, 4.15	

■■ BLOC 3 — Élaborer des Structures de Commande

Projet #08	Régulateur PID de température avec Arduino	■ 3 semaines
Contexte	Contrôle la température d'une enceinte (boîte isolante + résistance chauffante) avec un régulateur PID programmé sur Arduino. Utilise un thermocouple ou thermistance comme capteur de retour.	
Objectifs	• Identifier le modèle du système (réponse indicielle) • Tuner le PID (méthode Ziegler-Nichols puis ajustement fin) • Implémenter anti-windup • Comparer en boucle ouverte vs fermée	
Stack	Arduino Uno, résistance chauffante, thermistance/thermocouple, PWM	
Livrable	Démon PID fonctionnel + rapport avec courbes de réponse	
■ Bonus	Implémenter un régulateur PID adaptatif avec gain scheduling	
ACs SNI	AC 3.1, 3.2, 3.3, 3.7, 3.8, 3.9	

Projet #09	Bras robotique 3 degrés de liberté avec Simulink	■ 7 semaines
Contexte	Modélise un bras robotique 3-DDL en MATLAB/Simulink. Simule la cinématique directe et inverse, synthétise des lois de commande (PID, retour d'état) et valide sur un prototype Arduino.	
Objectifs	• Modéliser la cinématique directe et inverse • Simuler sous MATLAB/Simulink • Synthétiser un correcteur par retour d'état • Valider expérimentalement sur prototype servo-moteurs	
Stack	MATLAB/Simulink, Arduino, servomoteurs SG90, python pour visualisation	
Livrable	Modèle Simulink + prototype robotique + rapport de validation	
■ Bonus	Ajouter un capteur de force et contrôle d'impédance	
ACs SNI	AC 3.4, 3.7, 3.8, 3.14, 3.15, 3.16	

Projet #10	Drone autonome avec contrôle de stabilité embarqué	■ 12 semaines
Contexte	À partir d'un kit de drone (frame + ESC + moteurs), implémente le contrôleur de vol (algorithme PID multi-axe) sur STM32. Utilise MPU6050 pour attitude estimation (filtre complémentaire ou Kalman).	
Objectifs	• Lire et fusionner données IMU (gyro + accéléro + magnéto) • Implémenter filtre de Kalman ou complémentaire pour attitude • Synthétiser PID Roll/Pitch/Yaw • Tester en boucle fermée sur banc de test sécurisé	
Stack	STM32, MPU9250, ESC BLHeli, moteurs brushless, C embarqué (HAL/LL)	
Livrable	Contrôleur de vol embarqué + rapport d'identification et validation	
■ Bonus	<i>Ajouter mission autonome simple (maintien altitude par baro)</i>	
ACs SNI	AC 3.14, 3.15, 3.16, AC 1.8, 4.15	

■ BLOC 4 — Développer un Système Informatique Embarqué		
Projet #11	Plateforme domotique locale avec Raspberry Pi	■ 3 semaines
Contexte	Crée une maison connectée local-first : contrôle des lumières, température, capteur de présence, tout piloté via une interface web hébergée sur le Raspberry Pi. Pas de cloud requis.	
Objectifs	• Installer Home Assistant sur Raspberry Pi • Intégrer capteurs via GPIO (PIR, DHT22, relais) • Créer des automatisations (si temp > 25°C → ventilateur ON) • Interface web accessible sur réseau local	
Stack	Raspberry Pi 4, Home Assistant, Python, GPIO, capteurs variés	
Livrable	Plateforme domotique fonctionnelle avec 5+ automatisations	
■ Bonus	<i>Ajouter reconnaissance vocale locale avec Vosk (offline)</i>	
ACs SNI	AC 4.1, 4.3, 4.4, 4.12	

Projet #12	Système embarqué temps réel avec FreeRTOS sur STM32	■ 6 semaines
Contexte	Programme un STM32 sous FreeRTOS pour gérer plusieurs tâches concurrent : acquisition de capteurs (haute priorité), communication UART (moyenne), affichage LCD (basse). Respecte les contraintes temps réel.	
Objectifs	• Créer et gérer les tâches FreeRTOS (priorités, délais) • Utiliser les mutex et sémaphores pour la synchronisation • Mesurer les temps d'exécution (timer hardware) • Analyser la CPU load et les débordements de pile	
Stack	STM32F4 Discovery, FreeRTOS, STM32CubeIDE, oscilloscope	
Livrable	Application RTOS documentée + analyse des performances temps réel	
■ Bonus	<i>Implémenter watchdog et gestion des pannes matérielles</i>	
ACs SNI	AC 4.7, 4.8, 4.15, 4.16	

Projet #13	Compteur de personnes IoT avec traitement d'image Edge	■ 5 semaines
Contexte	Installe une caméra à l'entrée d'une pièce. Détecte et compte les passages de personnes avec un algorithme léger embarqué sur ESP32-CAM ou Raspberry Pi. Envoie les stats en MQTT vers un dashboard.	
Objectifs	• Configurer ESP32-CAM ou Raspberry Pi + caméra HQ • Implémenter détection légère (MobileNet SSD ou HOG) • Protocole MQTT pour envoi vers broker Mosquitto • Dashboard Grafana avec compteurs journaliers	
Stack	ESP32-CAM ou Raspberry Pi, OpenCV, MQTT (Mosquitto), Grafana	
Livrable	Système fonctionnel + stats de performance (précision, FPS, consommation)	
■ Bonus	<i>Ajout anonymisation (flou visage) pour conformité RGPD</i>	
ACs SNI	AC 4.12, 4.13, AC 2.10, 2.11	

Contexte	Conçois un bracelet connecté mesurant l'ECG (AD8232) et l'activité physique (MPU6050) sur Arduino Pro Micro. Transmet en temps réel via BLE vers une app Android. Détecte les anomalies cardiaques simples.
Objectifs	• Acquérir signal ECG propre (filtrage 50Hz secteur, haute fréquence) • Détecter les pics R (algorithme Pan-Tompkins embarqué) • Implémenter pile BLE (nRF52840 ou HM-10) • App Android simple pour visualisation temps réel
Stack	Arduino Pro Micro + AD8232 + MPU6050, module BLE, Android (MIT App Inventor ou Flutter)
Livrable	Prototype wearable + app mobile + rapport technique complet
■ Bonus	Classifier arythmies simples avec modèle TinyML embarqué
ACs SNI	AC 1.8, AC 2.4, AC 4.12, 4.13, 4.14, 4.15

Ressources matériel : Mouser.fr, Farnell, Gotronic, Adafruit · Datasets : Copernicus (satellite), Kaggle (signal/vibration) · Frameworks embarqués : STM32CubeIDE, PlatformIO, Arduino IDE · Référentiel SNI RNCP35711.